

DESKTOP COMPANION

A PROJECT REPORT

Submitted by
Akash Patil
Akash Deshmane
Yash Kodwate

in partial fulfillment for the award of the degree of

Bachelor of Technology

IN

Computer Science and Engineering



**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING KOLHAPUR
(EMPOWERED AUTONOMOUS)**

**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING KOLHAPUR
(EMPOWERED AUTONOMOUS)**

CERTIFICATE

This is to certify that the Seminar/ Project report entitled, **“Desktop Companion”** submitted by **“Akash Patil (A50), Akash Deshmane (A51), Yash Kodwate (A65)”** in partial fulfillment for the award of the degree of **“Bachelor of Technology”** in **“Computer Science and Engineering”** at Kolhapur Institute of Technology's College of Engineering Kolhapur (Empowered Autonomous), Kolhapur, Maharashtra, INDIA, is a record of his / her own work carried out under my / our supervision and guidance.

SIGNATURE

Prof. Dr. Lingaraj Hadimani
HEAD OF THE DEPARTMENT

SIGNATURE

Mr. Anesh Kshirsagar
Assistant Professor

**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING KOLHAPUR
(EMPOWERED AUTONOMOUS)**

DECLARATION

I hereby declare that the Seminar/ Project entitled, **“Desktop Companion”** submitted to Kolhapur Institute of Technology's College of Engineering Kolhapur (Empowered Autonomous), Kolhapur, Maharashtra, INDIA in the partial fulfillment of the award of the Degree of **“Akash Patil (A50), Akash Deshmane (A51), Yash Kodwate (A65)”** in **“Computer Science and Engineering”** is a bonafide work carried out by me. The material contained in this Seminar/ Project has not been submitted to any University or Institution for the award of any degree.

Akash Patil (A50)
Akash Deshmane (A51)
Yash Kodwate (A65)

Place: Kolhapur
Date:

**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING KOLHAPUR
(EMPOWERED AUTONOMOUS)**

ACKNOWLEDGEMENT

All the thanksgiving goes here.....

.....

.....

.....

Akash Patil (A50)
Akash Deshmane (A51)
Yash Kodwate (A65)

Place: Kolhapur
Date:

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	
1.	INTRODUCTION	
	1.1 GENERAL	
	1.2 PROBLEM STATEMENT	
	1.3 OBJECTIVES	
	1.4 SCOPE OF THE PROJECT	
2.	LITERATURE REVIEW	
3.	SYSTEM DESIGN	
	3.1 SYSTEM ARCHITECTURE OVERVIEW	
	3.2 BACKEND DESIGN	
	3.3 FRONTEND DESIGN	
	3.4 DATA FLOW	
4.	AI PROCESSING AND INTENT RECOGNITION	
	4.1 GENERAL	
	4.2 LLM BASED APPROACH (LLAMA 3.3)	
5.	IMPLEMENTATION	
	5.1 DEVELOPMENT TOOLS & ENVIRONMENT	
	5.2 BACKEND IMPLEMENTATION	
	5.3 FRONTEND IMPLEMENTATION	

6.	RESULTS AND DISCUSSION
	6.1 SYSTEM OUTPUT
	6.2 CATEGORIZATION ACCURACY
	6.3 DASHBOARD SCREENSHOTS
7.	CONCLUSION
	7.1 CONCLUSION
	7.2 FUTURE SCOPE

Abstract

The Desktop Companion is an AI-powered voice assistant designed to provide hands-free interaction with a computer system using natural voice commands. The system integrates speech recognition, large language models (LLMs), task planning, and automated execution to create an intelligent desktop automation platform. The assistant continuously listens for a wake word, converts speech into text, processes the user's intent using an LLM, and executes the required task such as opening applications, performing web searches, or controlling desktop functions.

The project uses technologies such as Python, FastAPI, Vosk, Deepgram and Llama-based language models to build a modular and scalable architecture. The backend handles voice processing and task execution, while the frontend provides monitoring and interaction capabilities. The proposed system demonstrates how AI can improve user productivity, accessibility, and human-computer interaction through voice-first interfaces.

The system is designed with a layered architecture consisting of wake word detection, speech-to-text processing, intent recognition, planning, execution, and text-to-speech response generation. Experimental results show that the assistant can accurately recognize commands and respond efficiently in real-time environments. Future enhancements may include memory support, multilingual interaction, cross-platform compatibility, and offline AI processing.

1. INTRODUCTION

1.1 General

Artificial Intelligence has transformed the way humans interact with machines. Traditional computer interaction methods rely heavily on keyboards, mice, and graphical interfaces, which may not always provide a natural or efficient user experience. Voice assistants have emerged as an advanced solution that enables users to communicate with systems using natural language. Popular assistants such as Siri, Alexa, and Google Assistant demonstrate the growing demand for voice-based human-computer interaction.

The Desktop Companion project aims to develop an AI-powered desktop voice assistant capable of understanding user commands and performing tasks automatically. The assistant operates using a continuous listening mechanism that activates after detecting a predefined wake word. Once activated, the system converts the user's speech into text, processes it using a Large Language Model (LLM), determines the user's intent, and performs the requested operation.

The project combines several modern technologies including speech recognition, natural language processing, AI-based reasoning, and desktop automation. The system is designed using a modular architecture where each component performs a dedicated task. This separation improves maintainability, scalability, and future enhancement possibilities.

The main objective of the Desktop Companion is to create an intelligent assistant that simplifies desktop operations and improves user productivity through natural voice interaction.

1.2 Problem Statement

Modern computer systems require users to interact through multiple manual operations such as typing commands, opening applications, searching the internet, and managing files. These repetitive interactions consume time and reduce efficiency, especially when multitasking. Existing desktop assistants are often limited by platform restrictions, dependency on cloud services, lack of customization, or high hardware requirements.

Many voice assistants are optimized for smartphones or smart home devices rather than desktop environments. Additionally, traditional systems often fail to understand complex natural language instructions or provide flexible task execution. Some systems also raise privacy concerns because voice data is continuously transmitted to external servers.

There is a need for an intelligent desktop assistant that can:

- Understand natural voice commands
- Execute desktop-related tasks efficiently
- Operate with modular and customizable architecture

- Integrate AI reasoning using modern LLMs
- Support real-time interaction
- Provide scalability for future features

The Desktop Companion project addresses these issues by developing a voice-first AI assistant capable of performing intelligent desktop automation while maintaining flexibility and extensibility.

1.3 Objectives

The major objectives of the project are as follows:

1. To develop an AI-powered desktop assistant capable of voice interaction.
2. To implement wake word detection for hands-free activation.
3. To integrate Speech-to-Text (STT) technology for converting spoken input into text.
4. To use a Large Language Model (LLM) for intent recognition and response generation.
5. To design a planner-executor architecture for task automation.
6. To enable desktop operations such as opening applications and performing web searches.
7. To provide voice responses using Text-to-Speech (TTS) technology.
8. To build a modular and scalable backend system using FastAPI and Python.
9. To improve user productivity through intelligent automation.
10. To create a foundation for future AI-based desktop interaction systems.

1.4 Scope of the Project

The scope of the Desktop Companion project includes the development of a real-time voice-controlled assistant for desktop environments. The system focuses on improving user interaction with computers by enabling natural language communication and automated task execution.

The project includes the following functionalities:

- Wake word detection using Vosk
- Speech recognition and transcription
- AI-based intent understanding using LLMs
- Task planning and execution
- Voice-based responses using Deepgram TTS
- Desktop automation features
- Web search integration
- Modular backend architecture

The project is limited to basic desktop automation and command execution. Advanced functionalities such as emotional AI, advanced memory systems, complete offline intelligence, and full operating system control are beyond the current scope but may be implemented in future versions.

The system mainly targets educational, research, productivity, and accessibility applications. It can also serve as a base platform for future AI assistants and smart automation systems.

2: LITERATURE REVIEW

Voice assistants and AI-powered automation systems have gained significant attention in recent years due to advancements in artificial intelligence, natural language processing, and machine learning. Researchers and technology companies have developed various intelligent assistants capable of understanding human speech and automating digital tasks.

Early voice assistants primarily relied on rule-based systems that required predefined commands. Modern systems now use machine learning and deep learning models to understand natural language and context-aware instructions. Technologies such as Speech-to-Text (STT), Text-to-Speech (TTS), and Large Language Models (LLMs) have significantly improved the capabilities of intelligent assistants.

Several existing systems such as Amazon Alexa, Google Assistant, Siri, and Cortana demonstrate the effectiveness of voice-controlled interaction. However, many of these systems are cloud-dependent and restricted to specific ecosystems. Researchers have also explored open-source solutions such as Vosk and Ollama for offline speech recognition and local AI processing.

Recent developments in transformer-based language models have enabled assistants to process complex user requests and generate intelligent responses. Models such as Llama, GPT, and Claude provide advanced reasoning capabilities that improve user interaction quality.

The Desktop Companion project builds upon these advancements by combining speech recognition, LLM reasoning, and desktop automation into a unified system specifically designed for desktop environments.

3. SYSTEM DESIGN

3.1 System Architecture Overview

The Desktop Companion follows a modular layered architecture where each component is responsible for a specific operation. The architecture consists of six major modules: Wake Word Detection, Speech-to-Text, LLM Processing, Planner, Executor, and Text-to-Speech.

The workflow begins when the wake word engine continuously listens for a trigger phrase. After activation, the Speech-to-Text module records the user's voice command and converts it into text.

The transcribed text is then sent to the LLM module, which identifies the intent and generates structured output.

The Planner module maps the identified intent to an appropriate action function, while the Executor module performs the actual task. Finally, the Text-to-Speech module converts the response into audio feedback for the user.

This layered approach improves maintainability, scalability, and system flexibility.

3.2 Backend Design

The backend is developed using Python and FastAPI. It manages the core functionalities of the system including audio processing, AI integration, task execution, and state management.

The backend contains multiple service modules such as:

- `wakeword.py` for wake word detection
- `stt.py` for speech transcription
- `llm.py` for AI processing
- `planner.py` for task routing
- `executor.py` for action execution
- `tts.py` for voice responses

The backend also maintains application states such as IDLE, LISTENING, THINKING, and EXECUTING. FastAPI provides lightweight and efficient API management for communication between components.

3.3 Frontend Design

The frontend provides a user interface for monitoring and interacting with the assistant. It may include features such as:

- Assistant status display
- Real-time transcription output
- System logs
- Voice interaction indicators
- Command history
- User controls for starting or stopping services

The frontend is designed to remain simple and lightweight while allowing efficient interaction with backend services. Future versions may include a fully interactive graphical interface and WebSocket-based live updates.

3.5 Data Flow

The data flow within the system follows a sequential pipeline architecture:

1. Wake word detection activates the system.
2. Speech input is captured through the microphone.
3. Audio is converted into text using Speech-to-Text processing.
4. The text command is sent to the LLM for intent recognition.
5. The Planner identifies the required action.
6. The Executor performs the requested task.
7. The result is returned to the Text-to-Speech module.
8. Audio feedback is delivered to the user.

This workflow ensures smooth real-time communication between all modules.

4. AI PROCESSING AND INTENT RECOGNITION

4.1 General

AI processing is the core functionality of the Desktop Companion. The system uses natural language understanding to identify user intentions and generate appropriate responses. Unlike traditional command-based systems, the assistant can understand flexible human language patterns.

The AI module processes the transcribed text and extracts meaningful information such as user intent, parameters, and response messages. This enables the assistant to perform intelligent actions rather than relying solely on predefined commands.

4.2 LLM-Based Categorization using Llama 3.3

The project uses a Llama-based Large Language Model for natural language understanding and response generation. The LLM receives the user command along with a system prompt that enforces structured JSON output.

The generated JSON typically contains:

- Intent type
- Action parameters
- Response message
- The planner reads the generated intent and routes it to the corresponding executor function. This design separates AI reasoning from execution logic, improving flexibility and maintainability.

- The use of Llama 3.3 enables efficient reasoning, contextual understanding, and customizable AI behavior.

5. IMPLEMENTATION

5.1 Development Environment and Tools

The project is developed using the following tools and technologies:

Technology	Purpose
Python	Core programming language
FastAPI	Backend framework
Vosk	Wake word and speech recognition
Deepgram	Text-to-Speech synthesis
Ollama / LLM APIs	AI processing
PyAudio / sounddevice	Audio input handling
asyncio	Asynchronous task handling
JSON	Structured data communication
Git	Version control

The development environment includes Python 3.11+, VS Code, and virtual environments for dependency management.

5.2 Backend Implementation

The backend implementation focuses on integrating all AI and automation modules into a unified workflow.

Major implementation steps include:

- Continuous wake word listening
- Recording microphone input
- Converting audio into text
- Sending text to the LLM
- Parsing structured JSON responses
- Routing tasks through the planner
- Executing desktop actions
- Returning voice responses

Threading and asynchronous programming techniques are used to improve responsiveness and real-time performance.

5.3 Frontend Implementation

The frontend implementation includes interfaces for displaying system states, transcripts, and logs. Communication between frontend and backend is managed through APIs and future WebSocket integration.

The frontend ensures that users can monitor assistant activity and interact with the system efficiently.

6. RESULTS AND DISCUSSION

6.1 System Output

The Desktop Companion successfully performs voice-based interaction and desktop task automation. The assistant can detect wake words, process speech commands, understand user intent, execute actions, and provide voice feedback.

The system demonstrates smooth interaction flow and real-time response generation.

6.2 Categorization Accuracy

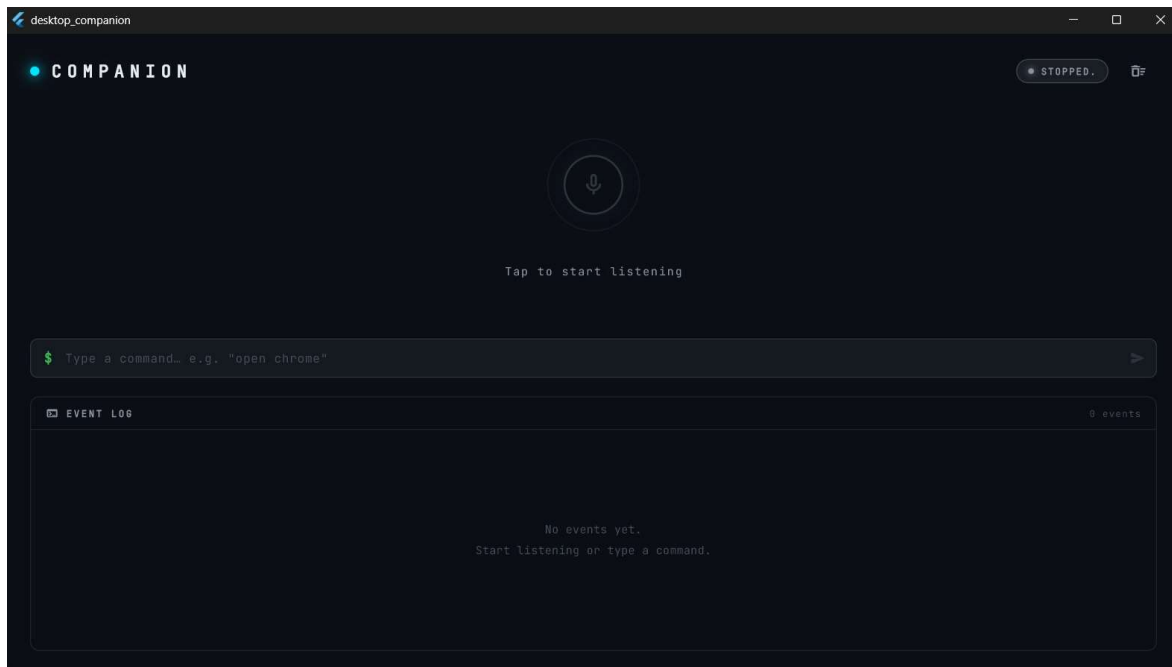
The LLM-based intent recognition system achieves high accuracy in understanding and processing common desktop commands such as opening applications, performing web searches, and executing system operations. By using a Large Language Model (LLM), the assistant can interpret natural language instructions more effectively compared to traditional rule-based systems. The overall performance of the system depends on several factors including microphone quality, speech clarity, pronunciation, background noise levels, and the quality of prompt engineering used for AI processing. Clear audio input and properly designed prompts significantly improve command recognition accuracy and response generation.

The modular architecture of the system further enhances reliability, scalability, and maintainability. Each component such as wake word detection, speech-to-text processing, LLM reasoning, task planning, execution, and text-to-speech operates independently. This separation allows individual modules to be optimized, updated, or replaced without affecting the entire system. As a result, the system becomes more flexible for future upgrades and easier to debug and maintain.

6.3 Dashboard Screenshots

This section contains screenshots of:

- Frontend dashboard



These screenshots demonstrate the practical working of the system.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Desktop Companion project successfully demonstrates the integration of artificial intelligence, speech recognition, and desktop automation into a single intelligent assistant system. The assistant provides natural voice-based interaction and automates common desktop tasks efficiently.

The modular architecture improves scalability and maintainability, while the use of LLMs enables flexible and intelligent command processing. The project highlights the growing importance of AI-driven human-computer interaction systems in modern computing environments.

7.2 Future Scope

Several meaningful extensions are possible for UniPay beyond the current demonstration:

- **Offline AI Processing Using Local LLMs:** Enables the assistant to work without internet while improving privacy, speed, and reliability.
- **Advanced Memory and Contextual Understanding:** Allows the assistant to remember previous conversations, user preferences, and ongoing tasks for smarter interaction.
- **Multilingual Voice Support:** Supports multiple languages for better accessibility and natural communication.
- **GUI-Based Desktop Companion:** Provides a graphical interface with real-time interaction, monitoring, and visual controls.
- **Cross-Platform Compatibility:** Allows the assistant to run on Windows, Linux, and macOS systems.
- **Smart Home Integration:** Enables control of IoT and smart home devices through voice commands.
- **Plugin Architecture for Third-Party Extensions:** Allows developers to add new features and integrations easily through plugins.
- **Personalized Voice Customization:** Enables users to customize the assistant's voice, tone, and interaction style.
- **Real-Time Learning and Adaptation:** Allows the assistant to continuously improve its responses and behavior based on user interactions.